INSTRUCTION FOLLOWING WITHOUT INSTRUCTION TUNING

John Hewitt, Nelson F. Liu, Christopher D. Manning, & Percy Liang
Department of Computer Science
Stanford University
{johnhew,nfliu,manning,pliang}@cs.stanford.edu

arXiv:2409.14254v1 [cs.CL] 21 Sep 2024

ABSTRACT

Instruction tuning commonly means finetuning a language model on instruction-response pairs. We discover two forms of adaptation (tuning) that are deficient compared to instruction tuning, yet still yield instruction following; we call this *implicit instruction tuning*. We first find that instruction-response pairs are not necessary: training solely on *responses*, without *any* corresponding instructions, yields instruction following. This suggests pretrained models have an instruction-response mapping which is revealed by teaching the model the desired distribution of responses. However, we then find it's not necessary to teach the desired distribution of responses: instruction-response training on narrow-domain data like poetry still leads to broad instruction-following behavior like recipe generation. In particular, when instructions are very different from those in the narrow finetuning domain, models' responses do not adhere to the style of the finetuning domain. To begin to explain implicit instruction tuning, we hypothesize that very simple changes to a language model's distribution yield instruction following. We support this by *hand-writing* a rule-based language model which yields instruction following in a product-of-experts with a pretrained model. The rules are to slowly increase the probability of ending the sequence, penalize repetition, and uniformly change 15 words' probabilities. In summary, adaptations made without being designed to yield instruction following can do so *implicitly*.

1 INTRODUCTION

Instruction tuning, finetuning on a broad distribution of responses (e.g., *Tiramisu is made by...*) conditioned on instructions (e.g., *Give me a recipe for tiramisu*), yields instruction following from language models for a wide range of instructions (Ouyang et al., 2022). Prior work has shown that instruction tuning is sample-efficient, requiring as few as 1000 broad-domain instruction-response pairs (Zhou et al., 2023) or a carefully crafted prompt and few-shot instruction-response examples (Lin et al., 2024). We take this a step further, exploring the idea that instruction following can be yielded from language models even *implicitly*, i.e., through methods not explictly designed to do so. We discover two forms of adaptation that perform *implicit instruction tuning*, being seemingly deficient compared to explicit instruction tuning: (1) *response tuning*, training on only responses, and (2) *single-task finetuning*, training on data from a narrow domain of goals, like poetry generation.

We first demonstrate that *response tuning*, training on responses alone without conditioning on their instructions—is sufficient to yield instruction following (Section 4). In particular, using the LIMA dataset (Zhou et al., 2023) for tuning, and evaluating on AlpacaEval 2, response-tuned models win roughly 43% of the time against a comparable instruction-tuned model, where equal performance would correspond to a 50% win rate. Response tuning provides no explicit information about the mapping from instructions to responses, only information about the distribution of desired responses. This suggests that an instruction-response mapping may be learned during pretraining, but all desirable responses are too low-probability to be generated. We support this, finding that pretrained models rank an instruction's real response higher than a random other instruction's response at the same rate as instruction-tuned models.

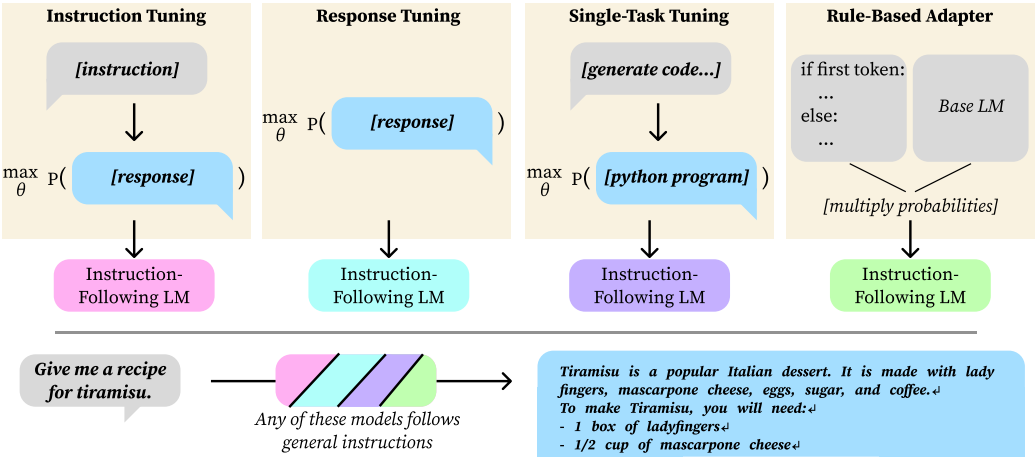

Figure 1: Instruction tuning trains a language model on responses conditioned on instructions. We find that (1) response tuning (estimating the probability of responses with no instructions), (2) single-task finetuning (e.g., code or poetry generation), and even (3) a simple rule-based adapter all yield language models with general instruction-following behavior.

From the response tuning results, it seems that the crucial part of instruction tuning is to teach the distribution of desirable responses. However, we find that this is not crucial either. Finetuning on single-task, narrow-domain data, like mapping English requests to Python snippets (MBPP; Austin et al., 2021), or generating poetry from poem titles, *also* yields broad instruction-following behavior (Section 5). That is, despite training only to generate only Python code or only poetry, models generate biographies or recipes when instructed. For example, a poetry-tuned Llama-2-7B wins 23.7% in AlpacaEval 2 head-to-head against an instruction-tuned Llama-2-7B, whereas the base model wins 2.4% of the time. Qualitatively, we find that single-task finetuned models only adhere to the finetuning distribution on instructions similar to those finetuned on, and exhibit general instruction following behavior for other instructions.

These two results motivate us to ask why instruction following is yielded by such a wide range of adaptations. While Lin et al. (2024) showed that relatively few token decisions change between pretraining and instruction following, it is not obvious that it's simple to determine which tokens to change. We hypothesize that very simple changes in conditional distributions can cause a language model to follow instructions. We validate this by hand-writing a rule-based language model with three rules that, when taken in a product with a pretrained language model, causes instruction following (Section 6). Our three rules are: slowly increasing the probability of ending the sequence, uniformly modifying 15 tokens' likelihoods (e.g., `I`, `<`, `should`), and penalizing token repetition. Our rule-based product-of-experts with Llama-2-7B wins 24.4% against instruction-tuned Llama-2-7B, compared to the base model's win rate of 2.4%.

In summary, our results show that adaptation methods not *intended* to yield instruction following may yet do so *implicitly*. When adapting language models for specific purposes, we might not expect that models would behave as general-purpose instruction following models on instructions dissimilar from those tuned on, but our results suggest that this will often be the case. Our rule-based adapter experiments begin to explain this, showing that very simple changes to a model's distribution—not obviously tied to following instructions—also yield instruction following.[1]

1.1 A NOTE ON PRETRAINING DATA

One critical question in the interpretation of our results is whether language models were instruction-tuned during pretraining, as is becoming common (Bi et al., 2024; Dominguez-Olmedo et al., 2024). If this is the case, our results become less surprising. To mitigate this, we experiment with two open-weights language models: Llama-2-7B and OLMo-7B-Feb2024. The Llama-2-7B model is

[1]Our code and data are available at https://github.com/john-hewitt/implicit-ins.

stronger than the OLMo-7B-Feb2024 model, but we have no guarantee against intentional instruction tuning. OLMo-7B-Feb2024 is weaker, but no instruction-tuning data was *intentionally* included in its pretraining.[2] These two models' results lead to similar conclusions in our experiments.

2 Related Work

Designing instruction tuning datasets. A few years before instruction tuning became common, the idea of a single model performing a range of tasks was explored by DecaNLP (McCann et al., 2018). In the LLM era, stronger pretrained models led to more adoption. In this era, early datasets also transformed a wide range of existing NLP tasks into a single common format (Mishra et al., 2022; Wang et al., 2022; Wei et al., 2022; Sanh et al., 2022). The introduction of ChatGPT in October of 2022 shifted focus onto the construction of instruction tuning datasets that reflect goals users might be interested in, as opposed to NLP tasks. Some are relatively small, with the intuition to teach nothing fundamentally new during instruction tuning (Taori et al., 2023; Zhou et al., 2023). Others, which tend to be more performant, can be quite large and contain reasoning chains and other behaviors thought to be rare in pretraining (Ivison et al., 2023; Wang et al., 2023b;a; Yu et al., 2023); this has included considerable non-academic open efforts (Teknium, 2023).

Ablation studies on instruction tuning. Our work contributes to a set of discoveries concerning how *little* it takes to get language models to follow instructions. Taori et al. (2023) showed that finetuning a Llama model on 52,000 instruction-response pairs caused it to follow instructions to an extent that was surprising at the time. Zhou et al. (2023) showed that—with careful selection—one could use as little as 1,000 instruction-response pairs and achieve strong results. This strongly suggested that one does not need to cover all types of instructions during instruction tuning. Lin et al. (2024) then showed that, with careful prompting and in-context few-shot examples, a handful of examples could also lead to instruction following. The most similar work to ours is Kung & Peng (2023). They take the NaturalInstructions (Mishra et al., 2022) style of instruction, which includes a task description, an in-context example, and the data for the current example. They then remove either the task description or the in-context example (leaving the other), finding that performance degrades less than expected in either case. In the direction of *more* supervision, Shi et al. (2024) find that training to maximize the joint likelihood of instruction and response improves over optimizing for the conditional likelihood of the response.

Out-of-distribution generalization. Discussing out-of-distribution generalization is difficult to do precisely in the world of language pretraining, especially when we don't know what data went into the pretraining. Furthermore, it's often difficult to reason about the coverage of datasets with 10+ trillion tokens (Section 1.1). In computer vision and vision-language models, various studies have shown that pretraining is the singularly important component in robustness to distribution shift (Carmon et al., 2019; Miller et al., 2021; Awadalla et al., 2022). In language as well, pretraining on diverse data is known to lead to better out-of-distribution performance (relative to the finetuning dataset) (Hendrycks et al., 2020). When we finetune, for example, on only poetry and test on recipes, this is out-of-distribution to the finetuning, but not to the pretraining.

3 Experiment Setting

A neural language model $p_\theta(\mathbf{x})$ is a distribution over strings $\mathbf{x} \in \mathcal{V}^*$, where $\mathcal{V}$ is a finite vocabulary, and θ are learnable parameters in the neural network. Language models are trained to minimize the cross-entropy loss with a large corpus of text. These pretrained, or *base*, models serve as starting points, but they are typically adapted to directly respond to user queries.

Instruction tuning. Instruction tuning finetunes the parameters θ of a language model to adapt its behavior to respond to queries with relevant, helpful answers. Given a set of samples $D_{\text{ins}} = \{\text{instruction}_i, \text{response}_i\}_{i=1}^k$, of instructions and corresponding responses (each of which in $\mathcal{V}^*$),

[2] Public communication. https://github.com/allenai/dolma/issues/177.

instruction tuning optimizes for

$$\min_{\theta} \frac{1}{k} \sum_{i=1}^{k} -\log p_\theta(\text{response}_i \mid \text{instruction}_i). \tag{1}$$

Instruction formatting. In language model practice, the distinction between the instruction and the response—and thus what to compute the loss over or generate—is specified through formatting tokens in the input. We use the Tulu formatting of Wang et al. (2023b); Ivison et al. (2023); we present it here because instruction formatting may matter for how easy it is to yield instruction following behavior from language models:

```
BOS<|user|>
{instruction}
<|assistant|>
{response}EOS
```

Here, `BOS` and `EOS` are the model-specified special beginning- and end-of-sequence tokens. Though the semantics of the tags are suggestive of instruction-following, we keep this formatting to study the setting used in practice. In Appendix B, we test whether the semantics of the tags affect the results in this paper, finding that non-semantic tags like `<|A|>` and `<|B|>` do not change our conclusions.

Defining instruction-following behavior. In this work we draw a distinction between instruction-following behavior and non-instruction-following behavior. In reality there is a spectrum of better and worse responses, and no single boundary. To provide some level of systematicity, we use the following evaluation setting:

AlpacaEval vs. a comparable Instruction-Tuned model. We want to avoid trying to standardize how good a response must be in order to be considered instruction-following. Further, we would like to compare various methods' effects on language model behavior compared to doing standard instruction tuning. We measure the length-controlled head-to-head win rate of each of our models against a comparable instruction-tuned model according to the AlpacaEval LLM-as-a-judge framework. We find in practice that these win rates concentrate around low single digits for the sort of outputs of base models, and are higher, e.g., $> 10\%$, for models with reasonable responses.

Greedy Decoding. Greedily decoding from a language model lets us only observe the locally most likely decisions. We greedily decode from models to observe when instruction following responses are (locally) the most likely continuations according to the model.

4 Response Tuning Yields Instruction Following

In this section, we explore *response tuning*, finetuning models on *just* responses–without any corresponding instructions. One intuition for the effect of instruction-tuning compared to pretraining is that it teaches a language model that (1) the {`instruction`} part of the input is an instruction to be followed, not a pattern to be continued. For example, for a prefix *Give me a recipe for tiramisu*, the model should not continue with *Give me a recipe for cake.* And (2), it should teach that strings like the {`response`} part of the input are considered desirable responses for that instruction. Instruction tuning is just conditional probability estimation, $p(y \mid x)$. Response tuning, on the other hand, can only directly teach the marginal distribution of desirable responses, not their relationships to instructions: we optimize only for $p(y)$. So, we examine whether response tuning *implicitly* tunes for a good estimate of $p(y \mid x)$ when we only intentionally trained for $p(y)$.

4.1 Response Tuning

Method. Given a set of samples $D_{\text{ins}} = \{\text{instruction}_i, \text{response}_i\}_{i=1}^{k}$, of instructions and corresponding responses (each of which in $\mathcal{V}^*$), response tuning replaces the instruction string with the

What are the names of some famous actors that started their careers on Broadway?

Instruction-Tuned

Ivan Boyko is a Ukrainian-born Canadian actor. He is known for his work on the Broadway stage, in television, and in film. Boyko began his acting career in Canada, appearing in television movies and making guest appearances on television shows. He moved to New York City in 1994 to pursue a career in theatre, and soon landed a role in the Broadway production of The Grapes of Wrath. He went on to appear in several[...truncated]

Response-Tuned

Some famous actors that started their careers on Broadway include:

* Al Pacino
* Robert De Niro
* Dustin Hoffman
* Meryl Streep
* James Earl Jones
* Nathan Lane
* Bette Midler
* Julie Andrews
* Audra McDonald

Base Model (No Adaptation)

What are the names of some famous actors that started their careers on Broadway? - <|user|>
<|assistant|> What are the names of some famous actors that started their careers on Broadway? - <|user|>
<|assistant|> What are the names of some famous actors[...truncated]

How did US states get their names?

Instruction-Tuned

General
States in the United States are named after people, cities, trees, animals, and geographical features. Some states share the same name, like New York and New York (state). Others are totally unique, like Hawaii.

All 50 states, in alphabetical order, are:
* Alaska[...truncated]
States named after people
* Alabama[...truncated]

Response-Tuned

The states of the United States of America were named after people and places with a historical and cultural significance.

* Alabama: named after the Alabama River which in turn was named after the Alabama tribe of Native Americans.
* Alaska: named after the Russian explorer, Vitus Bering, who discovered the peninsula in 1741.
* Arizona: named[...truncated]

Base Model (No Adaptation)

<|user|>
<|assistant|>
<|user|>
<|assistant|>
<|user|>[...truncated]

Figure 2: Responses from response tuning, instruction tuning, and the base Llama-2-7B model.

Model	Tuning	Win Rate vs. Instruction Tuning
Llama-2-7B	None (Base)	2.4% ± 0.14%
	Response Tuning	43.3% ± 1.1%
OLMo-7B-Feb2024	None (Base)	4.7% ± 0.57%
	Response Tuning	43.7% ± 1.7%

Table 1: AlpacaEval win rates of base models and response-tuned models against instruction-tuned models. Standard deviation is reported after the ±. Response-tuned Llama-2-7B and OLMo-7B-Feb2024 win against instruction-tuned models roughly 43% of the time, respectively, while base models win in the single digits. A win rate of 50% would denote equal-quality models.

empty string, uses the same formatting we show in Section 3, and optimizes

$$\min_{\theta} \quad \frac{1}{k} \sum_{i=1}^{k} -\log p_{\theta}(\text{response}_i \mid [\text{empty string}]). \tag{2}$$

Experiments. We now compare instruction tuning to response tuning. For our adaptation dataset, we use LIMA (Zhou et al., 2023), which has 1,030 training examples. For our base pretrained models, we use the Llama-2-7B and OLMo-7B-Feb2024 language models. We finetune all parameters of the models. For hyperparameter selection, we use the AlpacaEval win rate against GPT-3.5-turbo on a held-out validation set that we developed for this paper (and will release). Our validation set, written partly by hand and partly by GPT-4, has various knowledge, translation, and administrative instructions, like *Plan me a two day vacation to the virtual world of Code Lyoko*; more details are in Appendix D. For each method, we train for 5, 7, 10, 15, or 20 epochs. After hyperparameter selection, we report averages and standard deviations over 5 training runs. We report the length-controlled win rate of base and response-tuned models against instruction-tuned models on the AlpacaEval test set. More hyperparameter details are provided in Appendix A.

Results. We find that response-tuned Llama-2-7B models achieve on average a 43.3% win rate against instruction-tuned Llama-2-7B models, compared to a 2.4% win rate for the base model against instruction-tuned models. For OLMo-7B-Feb2024, response-tuned models win 43.7% of the time against instruction-tuned models, compared to 4.7% for the base model. We provide an example from a response-tuned, instruction-tuned, and base Llama-2-7B model in Figure 2.

The behavior of response-tuned models is much closer to instruction-tuned models than that of base models for both the Llama-2-7B and OLMo-7B-Feb2024 base models. Instruction tuning consistently outperforms response tuning, but not massively. So, there is something to be gained from specifying instructions during adaptation, but it is not crucial in yielding a baseline level of instruction-following behavior.

	$\mathbb{E}$[**Response Ranking Capability**]	
	Base Models	**Instruction-Tuned Models**
Llama-2-7B	80.4%	77.4%
OLMo-7B-Feb2024	74.5%	74.3%

Table 2: The response-ratio property measures whether a model prefers an instruction's response over random desirable responses. This property holds in pretrained language models at least as well as in instruction-tuned models.

It is possible that the success of response tuning is from responses often starting by rephrasing the instruction, thus making the mapping simpler. In Appendix C, we estimate that a bit less than 10% of LIMA responses begin with rephrasing. We run instruction and response tuning on a transformed version of LIMA with rephrasing approximately removed, finding a win rate of 43.3% for response tuning on OLMo-7B-Feb2024, almost identical to the result here. We conclude that instruction rephrasing is not key to response tuning's success.

4.2 The Response Ranking Capability

The success of response tuning suggests that we don't need to teach base models an explicit mapping from instructions to responses. Under what conditions is this true, yet base models do not follow instructions? One possibility is that base models can rank a desired response for an instruction higher than a desired response for *another* instructions, but scores a string that is *not a desired response at all* higher than both.

To explore this, we propose the *response ranking capability*: assigning a higher likelihood to the right response for an instruction than to a desirable response for a random other instruction. For independent instruction-response pairs $(\text{instruction}, \text{response}) \sim D$ and $(\text{instruction}', \text{response}') \sim D$, and a model p_θ, the response ranking capability holds if

$$p_\theta(\text{response} \mid \text{instruction}) > p_\theta(\text{response}' \mid \text{instruction}). \tag{3}$$

Since both probabilities might be small, the response-ranking capability can hold even for models that don't follow instructions. With response tuning—increasing the probability of desirable responses—models with the response-ranking capability for many instructions may generate desirable responses. For the Alpaca training set, we compute the likelihood with which the response-ranking capability holds for pairs of instructions for a pretrained, LIMA instruction-tuned, and response-tuned model. Our results show that the capability holds for pretrained models to a similar extent as for instruction-tuned models (Table 2).

5 Single-Task Finetuning Yields Instruction Following

The success of response tuning suggests that models need not learn the instruction-response mapping, so maybe they just need to learn the distribution of desirable responses. To test this, we can train on a distribution of responses that is *bad* for most responses—like poems, or the English-and-math derivations of the Grade School Math dataset (Cobbe et al., 2021)—and see if models yet generate desirable responses for other kinds of questions. We call training on a narrow domain of problem—like generating poems or recipes from titles, or completing math word problems—single-task finetuning. Intuitively, single-task finetuning might lead models to either (1) exhibit the task behavior for any input, like generating math, or (2) "break" and output unpredictable low-quality outputs. Single-task finetuning is usually not explicitly intended to yield broad instruction following behavior from base models, so if it does, it does so implicitly.

5.1 Single-Task Finetuning

Method. This method is identical to instruction tuning (Section 3), except the distribution of inputs and outputs is changed. We again use the same formatting shown in Section 3.

Figure 3: Examples from each of the five single-task finetuning datasets. At the left of each dataset is the input that is conditioned on, and at the right is the output that is learned.

Tuning	**Win Rate vs. Instruction Tuning**	
	Llama-2-7B	**OLMo-7B-Feb2024**
None (Base)	2.4% ± 0.14%	4.7% ± 0.57%
MBPP	16.9% ± 0.70%	10.4% ± 1.0%
GSM	23.7% ± 0.74%	30.3% ± 0.6%
Poetry	22.9% ± 0.97%	21.9% ± 0.48%
Recipes	14.6% ± 0.81%	21.5% ± 0.86%
Chess	2.1% ± 0.36%	6.3% ± 1.1%

Table 3: AlpacaEval win rates of single-task finetuned models against instruction-tuned models on five datasets. A diverse set of single-task finetuning, from code generation to poetry, elicits substantial instruction following in language models. Standard deviation reported after ±.

Data. We use five single-task text-to-text datasets to test a range of behaviors and text properties. We use the **MBPP** English-to-Python dataset, which contains 374 short requests for Python code and the corresponding code (Austin et al., 2021). We use 1000 examples from the Grade School Math 8K (Cobbe et al., 2021) training dataset, which consist of mathematics word problems, and a corresponding English-and-math derivation that arrives at the answer to the problem (**GSM**). We format 1000 recipe strings from the Kaggle Food Recipes structured recipe dataset (**Recipes**), where the input is *Recipe for X* where X is the name of the dish, and each output is a recipe starting with a bulleted list of ingredients, followed by the instructions.[3] We format a dataset of 571 poems (**Poetry**), wherein the input is the string *Write a poem called X*, where X is replaced with the poem name, and the output is the poem.[4] We format a dataset of 1000 chess games in PGN notation (**Chess**), wherein the input is the string containing the ELO (player quality) ratings for two players, and the output is the moves played in a game between those players.[5] See Figure 3 for an input-output example from each dataset.

The five datasets are qualitatively different. For example, the poems in the poetry dataset are famous (like the Shakespeare example in Figure 3), and so may have been memorized by the language model already. The recipes all start with a hyphenated ingredient list. Chess games start largely in one of a very small number of chess moves (e.g., "1. e4").

Experiments. We compare single-task finetuning against LIMA instruction tuning. For each dataset, we use the same hyperparameter selection process as in Section 4, sweeping over five choices for the number of training epochs. As before, AlpacaEval win rates and standard deviations are computed over 5 independent training runs of both instruction-tuned and single-task models.

[3]Drawn from https://huggingface.co/datasets/Hieu-Pham/kaggle_food_recipes, which drew from https://www.kaggle.com/datasets/pes12017000148/food-ingredients-and-recipe-dataset-with-images?resource=download.

[4]Drawn from https://huggingface.co/datasets/merve/poetry.

[5]Drawn from https://huggingface.co/datasets/patrickfrank1/chess-pgn-games.

How did US states get their names?

MBPP-Tuned	GSM-Tuned	Poetry-Tuned	Recipe-Tuned	Chess-Tuned
Alabama: Named after the Alabama River which in turn was named after the Alabama tribe of Native Americans who lived in the area. Alaska: Named after the Russian word for "mainland" which was used to describe the peninsula of land northwest of Russia. Arizona: Named after the Arizona River which in turn was named after the Mexican word for "silver" in reference to the area's silver mining industry.[...truncated]	Some states' names are self-explanatory, like New York, Florida, and Texas. Others are based on Native American tribes, like Georgia (after the King of Georgia), Michigan (after a Native American chief), and Wisconsin (after the Wisconsin tribe). Still others are based on explorers, like Kentucky (after Col. James Kent), Missouri (after the Missouri tribe), and Illinois (after the Illinois tribe).[...truncated] #### 2	The names of the fifty states of the United States of America are generally derived from geographical features or from early settlers of the area. In some cases, the origin of a state's name is not known.	- Jared Rubin - Twitter: [@JaredRubin11] (https://twitter.com/JaredRubin11) - Github: [...truncated] - Connecticut: named after the Connecticut River which runs through the state - Delaware: named after Lord De La Warr, the first governor of Virginia - Florida: named after the Spanish word for "land of flowers" - Georgia: named after King George II of[...truncated]	1. e4 e5 2. Nf3 Nc6 3. Bc4 Bc5 4. d3 Nf6 5. O-O d6 6. c3 Bg4 7. a4 a6 8. b4 Ba7 9. h3 Bh5 10. g4 Nxg4 11. hxg4 Bxg4 12. Be3 Qf6 13. Bxa7 Bxf3 14. Kh2 Qh4+ 15. Kg1 0-1

Figure 4: Responses generated by single-task finetuned models for each of our five datasets. MBPP trains only on python snippet generation, GSM on math word problems, Poetry on poetry generation, Recipe on recipe generation, and Chess on chess game generation. Yet, except for Chess, the responses deviate from the single-task behavior towards reasonable responses.

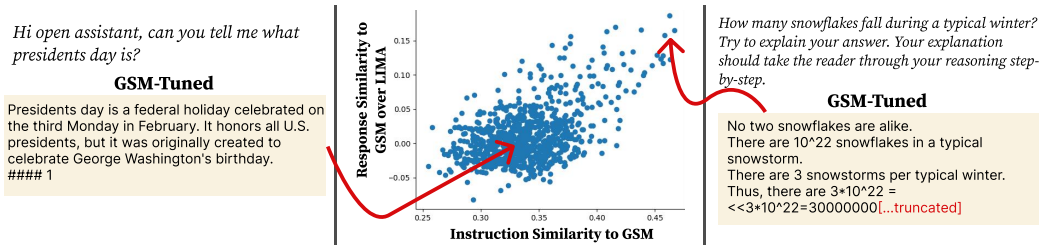

Figure 5: For a GSM-finetuned model, the similarity between a test-time instruction to the instructions in the GSM dataset (x-axis) plotted against the similarity between the model's generated response to GSM responses (minus the similarity of that response to LIMA broad responses). On the left, an example of an average-similarity instruction; note that the response is unlike GSM formatting, except for the telltale `####1`, which is how GSM formats its final answer. On the right, a very high-similarity instruction leads to GSM-like behavior.

Results. We find that finetuning both Llama-2-7B and OLMo-7B-Feb2024 on each single-task finetuning dataset except Chess lead to general instruction following behavior, with substantially higher win rates against the instruction-tuned model (Table 3) than the base model achieves.

For both OLMo-7B-Feb2024 and Llama-2-7B, finetuning on the GSM dataset leads to the highest AlpacaEval win rates. We provide examples of model outputs in Figure 4. We note, for example, that the Recipe-tuned example in Figure 3 starts with a list-formatting hyphen, like all of our recipes do, yet then proceeds to provide a somewhat coherent answer. For Chess, we find that almost all outputs are just chess games. We speculate that this is due in part to the very low entropy of the beginning sequences of chess games.

5.2 Model Response Adherence to Finetuning Constraints Depends on Instruction Similarity to Tuning Instructions

Each of our single-task finetuning datasets implicitly specify constraints—well-formed Python code for MBPP, poetry formatting—that are not met by the model's responses to other instructions, like for recipes. This is not necessarily bad—we don't necessarily want recipes in poetry formatting—but it's a striking instance of how finetuning for a desired behavior can cause neither (1) the finetuned behavior, nor (2) the base model's behavior, but instead (3) a general instruction following behavior similar to neither the base behavior nor the finetuning task.

We study how similarity of an instruction to the GSM instructions relates to the similarity of the model response to GSM responses versus general responses, expecting that at least the math-like instructions lead to GSM-like responses from the model. Using Nomic embedding (Nussbaum

et al., 2024) cosine similarities, we plot the similarity-to-GSM of each LIMA instruction against the similarity-to-GSM-over-LIMA of a GSM-tuned model's response to that instruction. The result is in Figure 5. We note qualitatively that for instructions that are very GSM-like, the model outputs adhere to the GSM style and particular mathematical notation it uses. For most instructions, however, we note that the outputs are only subtly affected by GSM: they have the ubiquitous GSM sequence-ending style of ending with four hashes and an integer answer, e.g., `####1`.

6 A 3-Rule Adapter for Instruction Following.

In this section, we provide steps towards understanding the surprising commonness of instruction following behavior. One appealing intuition is that the difference between a pretrained model's distribution and a corresponding instruction-following distribution is simple. Simple conditions should be relatively easy to meet, and might be met by a variety of disparate adaptation methods.

We provide direct evidence for the simplicity of a mapping from pretrained to instruction-following distribution by hand-writing a rule-based mapping with three rules. Prior work has shown that changes in distribution are somewhat sparse in token space: in a desirable response, Lin et al. (2024) found that 77.7% of token decisions would also have been made by a base language model. Still, 22.3% of tokens is not negligible, and even if it were, sparsity in the token space does not imply that it is simple to determine which tokens should change. For intuition, a weak chess engine may agree with a strong engine on, say, 95% of moves, but determining which 5% should be changed and how to change them could be very complicated.

6.1 The Rule-Based Response-Adapter

A product of distributions. To adapt a pretrained language model to follow instructions via a rule-based adapter, we choose our resulting model to have the form of a local product of distributions. For a word $w \in \mathcal{V}$ and prefix $\boldsymbol{x} \in \mathcal{V}^*$, a base model p_{base}, and our rule-based adapter language model p_{rules}, the final distribution p_a is:

$$p_a(w \mid \boldsymbol{x}) = p_{\text{base}}(w \mid \boldsymbol{x}) p_{\text{rules}}(w \mid \boldsymbol{x}) / Z(\boldsymbol{x}), \tag{4}$$

where the normalization term is $Z(\boldsymbol{x}) = \sum_{w \in \mathcal{V}} p_{\text{base}}(w \mid \boldsymbol{x}) p_{\text{rules}}(w \mid \boldsymbol{x})$.

Intuitively, a product of distributions is useful because it computes a soft AND function of the tokens that are likely under each distribution (as opposed to, e.g., a soft OR if one were to average the distributions). Put another way, it allows our rules to change the probabilities of the base model by multiplicative factors.

The rules. Each of our rules determines a score for each vocabulary item $w \in \mathcal{V}$; let $r(w, \boldsymbol{x})$ be the sum of all rules' scores for w. For example, for the vocabulary item `#`, it might score $+4$. Our rules distribution $p_{\text{rules}}(\cdot \mid \boldsymbol{x})$ is defined by computing the softmax over the vector of scores for all vocabulary items:

$$p_{\text{rules}}(\cdot \mid \boldsymbol{x}) = \text{softmax}\left(\left[r(w^{(1)}, \boldsymbol{x}); \cdots ; r(w^{(|\mathcal{V}|)}, \boldsymbol{x})\right]\right) \tag{5}$$

Our rules are as follows (with corresponding Python code in Listing 1, and weights in Table 8):

1. **Slowly upweight EOS.** Our first rule is to increase the score of the EOS token linearly with the response length, to favor shorter responses.
2. **Uniform token changes.** Our second rule is to uniformly change the probabilities of 15 words in the vocabulary at every token decision. For example, we massively reduce the probability of repeating tokens from the formatting, like the left angle bracket, or words "I" or "We" or "Should", which we found base models use to erroneously refuse to respond. The full list is found at Table 8.
3. **Encourage word diversity.** We compute the set of all tokens generated so far in the response, and add a penalty to generating any of them again.

Model	Rule-Based Model	Win Rate vs. Instruction Tuning
Llama-2-7B	None (Base)	2.4% $\pm$ 0.14%
	All Rules	24.4% $\pm$ 0.40%
	- EOS Rule (Rule 1)	10.4% $\pm$ 0.30%
	- Diversity Rule (Rule 3)	14.3% $\pm$ 0.58%
	- uniform token changes (Rule 2)	16.3% $\pm$ 0.25%

Table 4: Instruction-tuning win rates of a rule-based product Llama-2-7B model against the LIMA instruction-tuned Llama-2-7B model, and ablations of each of our three rules. Standard deviation reported after $\pm$.

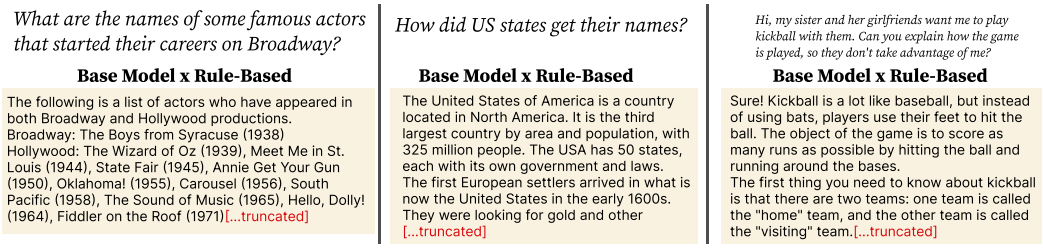

Figure 6: Responses from our rule-based language model product.

Experiments. We compare our rule-based ensemble with instruction tuning. We heuristically tuned the rule set and corresponding rule weights on our separate validation set. As before, we compute the AlpacaEval win rate on the AlpacaEval test set against our LIMA instruction-tuned models. Since there's no training for our rule-based model, we compute the average win rate and standard deviations of the one rule-based model over the 5 instruction-tuned model seeds.

Results. We find that our 3-rule model achieves win-rates against the LIMA-tuned Llama-2-7B model of 24.4%, roughly on par with the win rate of the best single-task finetuning setting. Our ablations show concretely that each of the three rules is necessary for achieving the win rate, at least in the context of the other two. We provide examples of the rule-based product model's output in Figure 6. The answers are coherent, but only the response to the kickball question really provides a good answer; we did not cherry-pick, and many other responses are more reasonable.

7 Conclusion

Instruction following is not only possible without explicit instruction tuning (or careful prompting), it is in some sense easy to stumble upon because some adaptations implicitly instruction-tune. Models already encode instruction-response mappings, since training on responses alone leads to instruction following. Training on, e.g., code, or math, or poetry, does not make language models generate those things when asked for, say, a recipe. This is great in some sense; language models continue to show astounding out-of-distribution performance. Except, in another sense, this shows it's perhaps surprisingly hard to get language models to change their behavior in general, since they are *so prone to just following instructions* outside the distribution of finetuning.

As a practical consequence, if a practitioner deploys an language model adapted to some specific task, they should not assume that the model will exhibit that tasks' behavior on inputs dissimilar to those trained on. Instead, they should put it through testing and safety trials as if they were releasing a general-purpose chatbot, since the finetuning may implicitly instruction-tune the model.

References

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021. arXiv:2108.07732.

Anas Awadalla, Mitchell Wortsman, Gabriel Ilharco, Sewon Min, Ian Magnusson, Hannaneh Hajishirzi, and Ludwig Schmidt. Exploring the landscape of distributional robustness for question answering models. In *Findings of EMNLP*, 2022.

Xiao Bi, Deli Chen, Guanting Chen, Shanhuang Chen, Damai Dai, Chengqi Deng, Honghui Ding, Kai Dong, Qiushi Du, Zhe Fu, et al. DeepSeek LLM: Scaling open-source language models with longtermism, 2024. arXiv:2401.02954.

Yair Carmon, Aditi Raghunathan, Ludwig Schmidt, John C Duchi, and Percy S Liang. Unlabeled data improves adversarial robustness. In *Proc. of NeurIPS*, 2019.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. arXiv:2110.14168.

Ricardo Dominguez-Olmedo, Florian E Dorner, and Moritz Hardt. Training on the test task confounds evaluation and emergence. *arXiv preprint arXiv:2407.07890*, 2024.

Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020. doi: 10.1038/s41586-020-2649-2. URL https://doi.org/10.1038/s41586-020-2649-2.

Dan Hendrycks, Xiaoyuan Liu, Eric Wallace, Adam Dziedzic, Rishabh Krishnan, and Dawn Song. Pretrained transformers improve out-of-distribution robustness. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 2744–2751, 2020.

Hamish Ivison, Yizhong Wang, Valentina Pyatkin, Nathan Lambert, Matthew Peters, Pradeep Dasigi, Joel Jang, David Wadden, Noah A Smith, Iz Beltagy, et al. Camels in a changing climate: Enhancing lm adaptation with Tulu 2, 2023. arXiv:2311.10702.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *CoRR*, abs/1412.6980, 2014. URL https://api.semanticscholar.org/CorpusID:6628106.

Po-Nien Kung and Nanyun Peng. Do models really learn to follow instructions? an empirical study of instruction tuning. In *Proc. of ACL*, 2023.

Bill Yuchen Lin, Abhilasha Ravichander, Ximing Lu, Nouha Dziri, Melanie Sclar, Khyathi Chandu, Chandra Bhagavatula, and Yejin Choi. The unlocking spell on base LLMs: Rethinking alignment via in-context learning. In *Proc. of ICLR*, 2024.

Bryan McCann, Nitish Shirish Keskar, Caiming Xiong, and Richard Socher. The natural language decathlon: Multitask learning as question answering, 2018. arXiv:1806.08730.

John P. Miller, Rohan Taori, Aditi Raghunathan, Shiori Sagawa, Pang Wei Koh, Vaishaal Shankar, Percy Liang, Yair Carmon, and Ludwig Schmidt. Accuracy on the line: on the strong correlation between out-of-distribution and in-distribution generalization. In *Proc. of ICML*, 2021.

Swaroop Mishra, Daniel Khashabi, Chitta Baral, and Hannaneh Hajishirzi. Cross-task generalization via natural language crowdsourcing instructions. In *Proc. of ACL*, 2022.

Zach Nussbaum, John X. Morris, Brandon Duderstadt, and Andriy Mulyar. Nomic Embed: Training a reproducible long context text embedder, 2024.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022. arXiv:2203.02155.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.

Victor Sanh, Albert Webson, Colin Raffel, Stephen Bach, Lintang Sutawika, Zaid Alyafeai, Antoine Chaffin, Arnaud Stiegler, Arun Raja, Manan Dey, M Saiful Bari, Canwen Xu, Urmish Thakker, Shanya Sharma Sharma, Eliza Szczechla, Taewoon Kim, Gunjan Chhablani, Nihal Nayak, Debajyoti Datta, Jonathan Chang, Mike Tian-Jian Jiang, Han Wang, Matteo Manica, Sheng Shen, Zheng Xin Yong, Harshit Pandey, Rachel Bawden, Thomas Wang, Trishala Neeraj, Jos Rozen, Abheesht Sharma, Andrea Santilli, Thibault Fevry, Jason Alan Fries, Ryan Teehan, Teven Le Scao, Stella Biderman, Leo Gao, Thomas Wolf, and Alexander M. Rush. Multitask prompted training enables zero-shot task generalization. In *Proc. of ICLR*, 2022.

Zhengyan Shi, Adam X. Yang, Bin Wu, Laurence Aitchison, Emine Yilmaz, and Aldo Lipani. Instruction tuning with loss over instructions, 2024. arXiv:2405.14394.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford Alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca, 2023.

Teknium. Openhermes 2.5: An open dataset of synthetic data for generalist llm assistants, 2023. URL https://huggingface.co/datasets/teknium/OpenHermes-2.5.

Guan Wang, Sijie Cheng, Xianyuan Zhan, Xiangang Li, Sen Song, and Yang Liu. Openchat: Advancing open-source language models with mixed-quality data, 2023a. arXiv:2309.11235.

Yizhong Wang, Swaroop Mishra, Pegah Alipoormolabashi, Yeganeh Kordi, Amirreza Mirzaei, Atharva Naik, Arjun Ashok, Arut Selvan Dhanasekaran, Anjana Arunkumar, David Stap, et al. Super-NaturalInstructions: Generalization via declarative instructions on 1600+ NLP tasks. In *Proc. of EMNLP*, 2022.

Yizhong Wang, Hamish Ivison, Pradeep Dasigi, Jack Hessel, Tushar Khot, Khyathi Chandu, David Wadden, Kelsey MacMillan, Noah A. Smith, Iz Beltagy, et al. How far can camels go? exploring the state of instruction tuning on open resources. In *Proc. of NeurIPS*, 2023b.

Jason Wei, Maarten Bosma, Vincent Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le. Finetuned language models are zero-shot learners. In *Proc. of ICLR*, 2022.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*, pp. 38–45, 2020.

Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. MetaMath: Bootstrap your own mathematical questions for large language models, 2023. arXiv:2309.12284.

Chunting Zhou, Pengfei Liu, Puxin Xu, Srini Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, Susan Zhang, Gargi Ghosh, Mike Lewis, Luke Zettlemoyer, and Omer Levy. LIMA: Less is more for alignment. In *Proc. of NeurIPS*, 2023.

A FINETUNING AND HYPERPARAMETER DETAILS

In this section, we describe hyperparameter choices and implementation details. In all experiments, we use core libraries, e.g., NumPy (Harris et al., 2020), PyTorch (Paszke et al., 2019), and Huggingface Transformers (Wolf et al., 2020). We're also indebted to the OpenInstruct repository (Wang et al., 2023b; Ivison et al., 2023), which we extended for this work.

For all Llama-2-7B models, after manual search, we choose 10^{-5} as a learning rate, and for all OLMo-7B-Feb2024 models, we choose $3 * 10^{-6}$. We use the Adam optimizer Kingma & Ba (2014).

Model	Tuning	Win Rate vs. Instruction Tuning	Std. Dev
Llama-2-7B	None (Base)	2.4%	0.14%
	Response Tuning	43.3%	1.1%
	Response Tuning (A/B tags)	41.8%	0.84%

Table 5: AlpacaEval win rates of instruction-tuned vs. response-tuned and base models. Standard deviation is reported by the error bars. Response-tuned Llama-2-7B and OLMo-7B-Feb2024 win against instruction-tuned models 43% and 36% of the time, respectively, while base models win in the single digits.

We use a cosine annealing rate to 0 learning rate, and 10% of the training consists of a linear warmup. When we sweep over epochs, we always take the final epoch, so the warmup and cosine decay have always completed for each model we evaluated. We fix a batch size of 64 across all experiments. We run experiments across A100 and A6000 NVIDIA GPU machines.

B Formatting Tags Ablation

One possible explanation for the success of response tuning in eliciting instruction-following behavior is that the semantics of instruction following are already encoded in the formatting tags `<|assistant|>` and `<|user|>` (see Section 3). To test this, we use the tags `<|A|>` and `<|B|>` instead, and compare response-tuned Llama-2-7B models with instruction-tuned Llama-2-7B models. We run exactly the experiment in Section 4.1, including re-running hyperparameter tuning, and averaging AlpacaEval scores across 5 independent pairs of model training runs.

With the new A/B tags, we find that response tuning wins over instruction tuning 41.8% of the time, compared to 43.3% of the time for the original user/assistant tags. So, it's possible that the semantics of the tags are somewhat useful, but not the cause of the improvement in win rate over the base model's 2.4%. The results are compared in Table 5.

C Instruction Rephrasing Removal Ablation

In this section, we detail an auxiliary experiment for response tuning in which we attempt to control for the effect of some responses beginning by *rephrasing the instruction*. Intuitively a response starting by rephrasing the instruction provides some supervision as to how to map from something *like* an instruction (the rephrasing) to the response. At test time when presented with an instruction, the model might start by rephrasing, and then be able to continue with the response.

To attempt to control for this, we first prompt GPT-4 to detect which responses in LIMA start by rephrasing the instruction. Out of 1030 examples, 99 are labeled as starting with rephrasing. A few examples are provided in Table 6. For these 99 examples, we then ask GPT-4 to re-write the response without starting with the rephrasing. A few examples of rephrasings are provided in Table 6. While there's no guarantee that these 99 are exactly all of the responses that begin with rephrasing, we looked at some of the decisions and rephrasings and found them reasonable.

We response-tune and instruction-tune OLMo-7B-Feb2024 using this transformed dataset and the hyperparameters from Section 4. We then run the AlpacaEval head-to-head comparison for these models. Intuitively, if the win rate for response tuning vs. instruction tuning are similar for this removed-rephrasing dataset to the original LIMA dataset, it is unlikely that instruction rephrasing is the cause of the success of response tuning.

Across five seeds, the average win rate of the OLMo-7B-Feb2024 model response tuned on the no-rephrasing LIMA dataset against the OLMo-7B-Feb2024 model instruction tuned on the no-rephrasing LIMA dataset is 43.3%, with a standard deviation of 5.3%. Although the standard deviation is curiously higher, the average win rate of 43.3% is comparable to the OLMo-7B-Feb2024 response tuning win rate in Section 4 of 43.7%. Hence, we conclude that rephrasing is likely not the primary cause of the success of response tuning, with the caveat that we have no guarantee that our transformed LIMA dataset gets rid of all instruction rephrasing.

Instruction	Original Response	Rephrased Response
What are the major concerns about planting trees to create carbon offsets?	Here are some common concerns about using tree planting projects to create carbon offsets: * The scheme actually has a net positive effect, both in the short and long-term.[...truncated]	* The scheme actually has a net positive effect, both in the short and long-term. [...truncated]
How does immersion passively help with learning a language?	There are a few reasons why immersion helps you learn a language without you actively trying: * You encounter the language more often. When you are immersed in a language, you don't go to class and do homework for a few hours a week, but rather use the language the entire day.[...truncated]	* You encounter the language more often. When you are immersed in a language, you don't go to class and do homework for a few hours a week, but rather use the language the entire day.[...truncated]
Reasons for being vegetarian or vegan other than ethical reasons?	There are multiple reasons why people are/become Vegetarian/Vegan apart from ethics, including but not limited to * Religion/Culture - most of the worlds Vegetarians reside in India, mainly due to culture/personal beliefs but often interlinked with some of their major religions[...truncated]	* Religion/Culture - most of the world's Vegetarians reside in India, mainly due to culture/personal beliefs but often interlinked with some of their major religions[...truncated]

Table 6: Examples of responses detected to start with rephrasing the instruction, and the transformed examples that are intended to no longer start with rephrasing the instruction.

Example Instruction from Validation Set
What is 17 multiplied by 24, divided by 8, and then subtract 3?
Summarize the story of 'Cinderella' in one sentence.
Given the list of numbers [34, 7, 23, 32, 5, 62], find the median.
Explain the significance of the theory of relativity in modern physics.
Write a persuasive paragraph on why electric cars are better for the environment than gasoline cars.
Describe the process of photosynthesis in detail, including the chemical equations involved.
Design a marketing campaign for a new eco-friendly product, including target audience, key message, promotional strategies, and budget allocation.
Translate the following legal document excerpt into German: 'The party of the first part agrees to indemnify and hold harmless the party of the second part from any and all liabilities, damages, and losses arising out of or in connection with this agreement.'
Write a scientific research proposal on the effects of microplastics on marine life, including hypothesis, methodology, and expected outcomes.

Table 7: Example instructions generated by GPT-4 for our validation set.

D Details on our Validation Set

To follow machine learning model development guidelines, we only run evaluations on the AlpacaEval test set after hyperparameter selection. We perform model selection and development on a separate validation set that we constructed partly by hand (without intentional reference to the AlpacaEval test set) and partly using GPT-4 as a co-generator.

Rule	Vocab Items (string)	Weight
Rule 1 (Upweight EOS)	`</S>` (EOS)	$\frac{(\text{length of response}) * 15}{250}$

Rule 2 (Uniform Token Changes)	`<`, `_<`, `|`
	`_I`, `I`
	`We`
	`What`
	`_should`
	`_*`, `_-`, `____`, `_#`, `_##`, `\n`, `!`
		-4
		-5
		-3
		-3
		-6
		+1

Rule 3 (Penalize Used Words)	$\{x \in \mathcal{V} \mid x \in \text{(response so far)}\}$	-1.5

Table 8: Rules and scores for our rule-based adapter.

Our validation set, which has 56 instructions, tests for a variety of behaviors. Here are some examples we wrote:

> What is Samyang Buldak ramen?
>
> Give the Penn Treebank constintuency parse for the sentence "The chef who ran to the store was out of food."
>
> Give a python function that sorts a list by absolute value of difference from 10.

In a single interactive session with GPT-4, we iteratively requested instructions for testing chatbots, starting with simple questions for "weak" chatbots and progressively asking for harder questions for "stronger" chatbots. Our intuition was to develop a small, cheap-to-evaluate set that would yet provide signal in distinguishing model quality across a range of model sizes and performances. It is unclear the extent to which this goal was accomplished beyond what a simple set of instructions would provide, but we did qualitatively find that the AlpacaEval results on this small validation set correlated reasonably well with intuitions when looking manually at responses. The instructions suggested by GPT-4 include outlining requests, translations of varying lengths and languages, among other things. We provide a few of these instructions in Table 7.

E Rule-Based Adapter Details

In this section we provide more details on our rule-based adapter language model. In Table 8, we provide a list of details about the scores for each word under each of our rules. In Listing 8, we write Python code implementing the forward pass of our language model implemented by the rules to demonstrate the simplicity. Some of the lines are to, e.g., format the output distribution properly to be combined with a HuggingFace Transformers (Wolf et al., 2020) language model.

Listing 1: Python Code for Rule-Based Language Model.

```
def forward(
    self,
    input_ids,
    attention_mask=None,
    position_ids=None,
    inputs_embeds=None,
    labels=None,
    use_cache=None,
    output_attentions=None,
    output_hidden_states=None,
    return_dict=True,
    past_key_values=None,
    cache_position=None,
):

    # Initialize scores
    output = torch.zeros(self.vocab_size).to(input_ids.device)
```

```python

    # Uniform biases
    # Formatting types "<", "_<", "|"
    output[29966], output[529], output[29989] = -4, -4, -4

    # Types "_I", "I", "We"
    output[306], output[29902], output[1334] = -5, -5, -3

    # Types "What", "_should"
    output[5618], output[881] = -5, -6

    # Types "_*", "_-", "___" for markdown
    output[334], output[448], output[1678] = 1, 1, 1

    # Types "_#", "_##", Newline for markdown
    output[396], output[444], output[13] = 1, 1, 1

    # Exclamation point "!" for positivity.
    output[29991] = 1

    # Find where the response starts
    assistant_start_tag = self.tokenizer(f"\n{ASSISTANT_TAG}\n")["input_ids"][-5:]
    user_start_tag = self.tokenizer(f"\n{USER_TAG}\n")["input_ids"][-5:]

    idlist = input_ids[0].tolist()
    first_token_index = next(
        (i + 5 for i in range(len(idlist)) if idlist[i : i + 5] == user_start_tag), None
    )

    # Penalize reusing words
    uniq_words = set(idlist[first_token_index:])
    for w in uniq_words:
        output[w] -= 1.5

    prefix_len = input_ids.shape[-1] - next(
        i + 5 for i in range(len(idlist)) if idlist[i : i + 5] == assistant_start_tag
    )

    # Increase probability of EOS
    # There's a bug here... there's no weight for indices 251-1023.
    # But in practice all the responses ended before 250.
    eos_range = (0, 250)
    if eos_range[0] < prefix_len < eos_range[1]:
        score = max(0, (prefix_len - eos_range[0]) / (eos_range[1] - eos_range[0]))
        output[self.eos_id] = score * 15
    # This never triggered; I'd get rid of it.
    if prefix_len > 1024:
        output[self.eos_id] = 100

    # Format the output to be combined with Llama LM
    output = output.unsqueeze(0)
    pad = torch.zeros(input_ids.shape[-1] - 1, self.vocab_size).to(output.device)
    output = torch.cat((pad, output), dim=0)
    output = output.unsqueeze(0).expand(input_ids.shape[0], -1, -1)

    return namedtuple("Result", ["logits"])(logits=output)
```